

QKF-Certifier: observation kernels and width-independent word proofs

Leonid Shcherbakov

13 September 2026

Preprint, version 2.0

Abstract

QKF studies how incomplete observations determine computations through forced rows, their kernels, and compatible quotients. This paper develops that viewpoint into a certificate architecture for word programs. A consumer specifies which distinctions a proof must retain; checked native facts label the observation classes; algebraic closure completes the required rows; and a finite closure certificate composes their actions across an arbitrary number of bit positions. The resulting interface can describe the action of an expression without reconstructing every intermediate word.

We retain the exact finite relation-matrix kernel and the complete nine-row characterization of the coordinatewise KnownBits fragment, and extend the account to contextual observations, guarded whole-word reconstruction, and signed mask-interval carriers. A descending repair pass uses a common-prefix observation with 49 certificate states; an ascending successor pass factors three physical phases into two carry observations and has an eight-state certificate. The upper helper computes an exact masked maximum with an explicit empty-case distinction. The lower helper preserves the joint carrier under a stated sign condition and need not return its exact minimum.

A self-contained Lean 4 pilot checks forced-row rules, the carry factor, finite gluing, and the successor result for every positive word width. Its scope is the explicit mathematical model and the modeled nonsign repair loop. The Python checkers, complete Java procedures, and full Graal constructor remain separate proof obligations. Research evidence includes 11 of 39 complete Nice to Meet You programs, 104 of 411 components, and two of five complete Graal mask results; these denominators describe different experiments. The release separates the coordinatewise CLI, broader research capsules, and Lean project, with exact source manifests and replay instructions.

1 Introduction

A word program may inspect a count, use a shift, propagate a carry, and finally return only a few facts about its result. A proof that reconstructs every intermediate value can retain substantially more information than the consumer needs. QKF asks which observations determine this particular action, and which of their values are forced by the supplied information and the operations already justified on the carrier.

This is Paper III of a series by Leonid Shcherbakov. Paper I develops initial semantics for incomplete actions, forced domains, kernel saturation, and completion [1]. Paper II studies equality visibility through finite ground interfaces and distinguishes congruence-proof depth from the size of sequential rewriting contexts [2]. The present paper uses those distinctions to organize executable proof objects. Native semantics supplies the facts to which the algebra applies; a quotient is useful only when its labels and its interaction with the consumer have been justified.

The first version described an exact finite algebraic kernel and a public KnownBits certifier whose final language was coordinatewise. Nine one-bit rows then suffice for a theorem at every positive width. Subsequent research has made the interface itself a computational object: counts can be replaced by their observed action, contexts can justify observations that fail unconditionally, and a protected row carrier can connect local repair decisions into a proof about a whole signed word. This revision develops that change of emphasis and incorporates the first Lean verification.

Contributions and stable claims. Claims K1–K5 retain their meanings: a finite ground partial core, exact relation-matrix closure, a conditional sparse representation contract, source-to-verdict replay, and the complete coordinatewise semantic quotient. New claims describe (K6) sufficient consumer observations and labeled reconstruction, (K7) forced observation rows and arbitrary-length gluing, (K8) the exact upper masked-bound contract, (K9) the conditional lower carrier-preservation contract, and (K10) the precisely delimited Lean pilot. Elementary quotient and induction arguments provide the foundations of K6–K7; their role here is to expose and check the obligations of the actual word interfaces.

Layer	Computation	Evidence in this revision
Finite algebraic core	Forced relations, central feedback, sparse coverage	Mathematical proofs and retained independent finite checks.
Coordinatewise CLI	Source-bound normalization followed by nine-row target checking	Versioned Python package; unchanged core semantics and rule catalogue.
Research interfaces	Contextual observations, complete word certificates, signed mask–interval refinements	Separate executable capsules with source-bound replay.
Lean pilot	Row soundness, carry factor, gluing, numerical successor theorem	Kernel-checked Lean source and audited assumptions.

These layers have different interfaces. The command-line package does not automatically acquire all research targets by being distributed beside them. The general relation backend is not silently invoked by every word checker. Paper II does not confer a minimum proof-depth guarantee on a heuristic normalizer. The explicit maps between the layers are part of the algorithm’s specification.

Version and source identity. This is manuscript version 2.0, prepared with software release candidate 0.2.0a1. The public baseline is v0.1.0a1, commit

f93561682319e8b911ed32c01583f2a496dc59fc.

The repository is <https://github.com/Len989/QKF-Certifier>. The coordinatewise parser, kernel, producer, and certificate implementation retain that baseline’s semantics; the new distribution adds the research and Lean profiles. Content manifests bind the candidate files without assuming that a future tag or DOI already exists. The coordinatewise certificate schema remains `qkf-rewrite-v3`; research and Lean certificates have their own identifiers.

The long-term research objective is an observation-driven proof architecture capable of taking over substantial tasks currently delegated to general SMT infrastructure. The immediate technical goal is narrower and testable: infer adequate interfaces from source semantics, compose them without losing the joint carrier, and make their successful conclusions independently replayable.

2 The finite partial core inherited from I and II

Let Σ be a finite finitary signature, E a finite set of ground equations, and $\mathcal{Q}$ a finite set of query terms. Constants have depth zero. Put

$$\begin{aligned} S &= \text{Sub}(E) \cup \text{Sub}(\mathcal{Q}), & S_D &= \{u \in S : \text{dep } u \leq D\}, \\ E_D &= \{s = t \in E : \max(\text{dep } s, \text{dep } t) \leq D\}. \end{aligned}$$

Let $\approx_D^S$ be congruence closure on the represented applications in S_D . Write $\equiv_D$ for the bounded *congruence-proof* equality of Paper II and $\equiv_E$ for full ground equality.

Proposition 2.1 (Partial core and evaluation, K1). *The quotient $\mathbf{P}_D = S_D / \approx_D^S$ is a well-defined partial algebra: a tuple has an operation value when it is represented by an application node, and its value is the parent class. Every total $M \models E_D$ induces an operation-preserving map*

$$\eta_M : \mathbf{P}_D \rightarrow M, \quad [u] \mapsto \llbracket u \rrbracket_M.$$

For active query terms s, t , their classes coincide exactly when $s \equiv_D t$. At $H = \max\{h(E), d_{\mathcal{Q}}\}$ this is equivalent to $E \vdash s = t$.

Proof. Congruence closure identifies the parents of any two represented tuples with componentwise equal arguments, so the partial operations are well defined. Every generating equation holds in M , and equality is preserved by its operations; hence evaluation factors through the quotient and preserves represented operations. The equality assertions are Paper II, Theorems 4.1 and 3.1, respectively. Empty query depth and empty axiom depth are taken to be zero. $\square$

This core precedes any choice of values for unrepresented tuples. A finite separating total model is obtained by assigning a default value to each unrepresented operation tuple and adding a default element to every empty active sort. That totalization is a certificate-producing construction, not additional canonical information in $\mathbf{P}_D$.

A positive derivation shows that two classes must merge. Conversely, a total model of E_D separating their labels shows that they cannot merge at that horizon. Paper II, Proposition 4.3 and Corollary 4.5, combine a chronological proof DAG at the first complete threshold with such a lower model. Their resource λ_E bounds all terms in a congruence proof. The different sequential resource λ_E^{rw} bounds whole intermediate rewrite terms. Neither is asserted to equal the number of normalization steps in the public certifier.

2.1 The same action example in all three papers

For the carrier $A = \{0, 1, 2, 3\}$, use the operation table

f	0	1	2	3
0	0	2	0	0
1	1	2	1	3
2	0	2	2	0
3	3	0	3	0

and one operator b , with $r_i = \alpha(b, i)$ and observations $r_0 = 0, r_2 = 2, r_3 = 0$. Include the native diagram and named-input compatibility. Paper I, Example 6.2, and Paper II, Section 5.1, give

$$\begin{aligned} \mathcal{I}/\equiv_1 &= \{\{0, r_0, r_3\}, \{1\}, \{2, r_2\}, \{3\}, \{r_1\}\}, \\ \mathcal{I}/\equiv_2 &= \{\{0, 2, r_0, r_1, r_2, r_3\}, \{1\}, \{3\}\}. \end{aligned}$$

For example, $f(0, 1) = 2$ and $f(3, 1) = 0$ yield $r_2 = f(r_0, r_1) = f(r_3, r_1) = r_0$. Likewise $f(1, 0) = 1$ and $f(1, 3) = 3$ yield $r_1 = r_3$. The canonical maps in Proposition 2.1 thus recover the five-to-three class change on exactly the named interface described in Paper I, Theorem 6.6.

This is an algebraic example, not a KnownBits program. Its role in III is to fix the meaning of a forced quotient and a finite interface before introducing specialized representations. Quotient-feedback rounds, syntactic depth, proof length, and semantic table size remain different quantities throughout.

3 Exact relation-matrix closure

3.1 Ground specification and labelled copies

Let A be a nonempty finite algebra, $|A| = n$, whose native operations have positive arity. Add a fresh binary symbol τ and observations $\Gamma \subseteq A^2 \times A$. The ground theory E_τ contains the complete named native diagram, the observed equations $\tau(a, b) = y$, and separate named-input compatibility. For each native k -ary g , include

$$\begin{aligned}\tau(g(a_1, \dots, a_k), b) &= g(\tau(a_1, b), \dots, \tau(a_k, b)), \\ \tau(b, g(a_1, \dots, a_k)) &= g(\tau(b, a_1), \dots, \tau(b, a_k)).\end{aligned}$$

All displayed arguments are carrier names. No universal identity for g on unnamed elements is imposed. The constants naming A are not nullary native operations required to be preserved by every slice.

Use copies $\Lambda = \{*\} \cup \{R_b : b \in A\} \cup \{C_a : a \in A\}$. A label (i, x) denotes

$$\ell_*(x) = x, \quad \ell_{R_b}(x) = \tau(x, b), \quad \ell_{C_a}(x) = \tau(a, x).$$

Two slice labels can represent one physical cell. Each relation $R_{ij} \subseteq A^2$ records proposed equalities between the labels in copies i and j .

3.2 The closure rules

Initialize diagonal reflexivity, physical-cell facts $(a, b) \in R_{R_b, C_a}$, and the facts $(a, y) \in R_{R_b, *}$ and $(b, y) \in R_{C_a, *}$ for every observation. Saturate the following rules:

- (R1) Converse: $(a, b) \in R_{ij}$ implies $(b, a) \in R_{ji}$.
- (R2) Composition: $(a, b) \in R_{ij}$ and $(b, c) \in R_{jk}$ imply $(a, c) \in R_{ik}$.
- (R3) Native compatibility: from $(a_l, b_l) \in R_{ij}$ for all $1 \leq l \leq k$, infer

$$(g^A(a_1, \dots, a_k), g^A(b_1, \dots, b_k)) \in R_{ij}.$$

- (R4) Local feedback: $R_{**} \subseteq R_{ii}$ for every copy i .
 - (R5) Context feedback: $b R_{**} d$ implies $(x, x) \in R_{R_b, R_d}$ and $(x, x) \in R_{C_b, C_d}$ for every x .
- Let R^* be the least closed matrix containing the initial facts.

Theorem 3.1 (Exact relation kernel, K2). *A fair saturation that returns only at a common fixed point terminates and computes R^* . For all interface labels,*

$$(x, y) \in R_{ij}^* \iff E_\tau \vdash \ell_i(x) = \ell_j(y).$$

*In particular $R_{**}^* = \theta_{E_\tau}^A$. The carrier values forced for $\tau(a, b)$ are exactly $\{y : (a, y) \in R_{R_b, *}^*\}$; a nonempty such set is one central-congruence class.*

Proof. There are $m = 1 + 2n$ copies and at most $m^2 n^2$ relation facts. Every productive rule application adds a fact. Finite monotone saturation therefore reaches the least fixed point; a worklist or complete-pass implementation must stop when no new fact is available.

For soundness, converse and composition express equality, and physical-cell facts identify the two names of the same term. Each copy preserves every named native operation by the diagram or compatibility equations. Thus native compatibility is sound between any pair of copies. Local and context feedback are congruence of τ in its varying and fixed arguments. All added facts are consequently ground consequences of E_τ .

For completeness, take any closed matrix R . On $V = \Lambda \times A$ put $(i, a) \sim (j, b)$ precisely when $(a, b) \in R_{ij}$. Reflexivity, converse, and composition make this an equivalence. Set $M = V/\sim$. On tuples represented in one copy, define

$$g^M([(i, a_1)], \dots, [(i, a_k)]) = [(i, g^A(a_1, \dots, a_k))].$$

If another representation of the same tuple uses copy j , native compatibility identifies the two proposed results. This defines a partial operation on classes; assign a fixed default class to all other tuples. Interpret carrier names by their central classes and put

$$\tau^M([(*, a)], [(*, b)]) = [(R_b, a)].$$

Local feedback makes the value independent of the representative of a ; context feedback does the same for b . Physical-cell facts give the equivalent column interpretation. Complete all remaining τ -tuples by a default class.

The named diagram, observations, and both compatibility families now hold by construction. Each interface label evaluates to its own class, so distinct R -classes are separated by a total model of E_τ . Applying this construction to R^* proves completeness. The assertion about forced values follows by transitivity with central equality. $\square$

The bound counts possible facts, not the running time of an inefficient repeated composition scan. For fixed native arities an explicit implementation is finite and polynomial in the explicitly represented carrier and rule-instance space; no polynomial bound in a succinct bit-width parameter follows. A native nullary operation would require additional zero-premise cross-copy facts and is excluded from this specification.

Reference algorithm. Initialize the labelled relations; repeatedly apply feedback, native compatibility, and full relational composition, inserting converse facts with each insertion. Return only when a complete pass makes no change. The supplement implements this deliberately transparent oracle and compares its entire carrier/cell partition with independent ground subterm closure. A finite resource limit must return unresolved, never an unfinished matrix presented as exact.

3.3 Certificates and two necessary closure mechanisms

Corollary 3.2 (Checking a proposed exact partition). *A proposed interface partition is exact if (i) generators of each of its blocks have checked derivations from the initial facts and rules, and (ii) the partition contains every initial fact and is closed under every rule.*

Proof. The derivations give $R \subseteq R^*$. Closure and the initial facts give $R^* \subseteq R$, by leastness. Equivalently, the first check excludes unjustified equalities and the second supplies the separating model from Theorem 3.1. $\square$

This algebraic certificate contract is separate from `qkf-rewrite-v3`, which certifies word-expression rewrites.

Example 3.3 (Composing generators is insufficient). In the one-bit KnownBits join-semilattice, write U for unknown and Z for known zero, so $U \vee_A Z = U$. Let $G = \{(U, U)\}$ and $H = \{(U, Z), (Z, U)\}$. The join-closure of H also contains (U, U) . Hence the composition of the two closed relations contains both (U, Z) and (U, U) . Composing only the displayed generators gives $\{(U, Z)\}$, whose join-closure still omits (U, U) . Rule R2 must quantify over the full represented relations, including derived intermediate witnesses.

Example 3.4 (Feedback determines additional cells). Take the two-element join-semilattice and supply both $\tau(0, 0) = 0$ and $\tau(0, 0) = 1$. The central carrier collapses. With feedback, all four named input cells become equal to that central class. If feedback is omitted while the other rules are saturated, only the observed cell is connected to the center. This finite example is checked in the supplement.

The same model proof extends to a fixed p -ary τ using $1 + pn^{p-1}$ copies, one for each varying coordinate and fixed context. All views of a physical cell must be identified, and feedback must cover every context coordinate. The implementation and differential results reported here concern the binary case.

4 Sparse semilattice representations

Suppose now $A = (A, \vee_A)$ is a finite join-semilattice and τ is binary. The relation matrix can be represented by global equality components of labelled endpoints instead of storing every R_{ij} explicitly. A missing physical cell still denotes a two-endpoint component, one in its row and one in its column. It is implicit, not absent.

For the current central quotient $Q = A/\theta$, materialize all central endpoints. Materializing either view of a cell also materializes its other view and identifies them. For each used copy i , maintain its exact local congruence $\kappa_i \in \text{Con}(Q)$ and materialize every affected nontrivial local class. This guarantees that an unmaterialized cell has not silently acquired further equalities.

Lemma 4.1 (Compatibility coverage). *After exact local closure, all binary native-compatibility obligations are covered by:*

- (i) *each pair of distinct explicit global components, compared in every copy in which both occur;*
- (ii) *each explicit component and implicit physical cell, compared across that cell's row and column when both occur in the explicit component.*

Repeated components and pairs of distinct implicit cells create no further cross-copy obligation.

Proof. Representatives of one component inside one copy are interchangeable modulo κ_i . This reduces two distinct explicit components to one pair of representatives per common copy. Repeating one component gives its own class by idempotence; local closure handles different representatives of that component. Commutativity allows unordered component pairs.

An implicit component occurs in exactly one row and one column. Thus its only possible nontrivial comparison with an explicit component is (ii). Two distinct implicit cells cannot share both their row and column, so they have at most one common copy and create no cross-copy identification. Repeating one implicit cell is again covered by idempotence. Local compatibility in a single copy is already included in exact local congruence closure. $\square$

Theorem 4.2 (Sparse refinement contract, K3). *An algorithm that preserves the representation above, fairly saturates the two cases of Lemma 4.1, and performs full central quotient feedback computes the same interface partition as Theorem 3.1, provided it reports success only after a common fixed point is reached.*

Proof. Every materialization, local merge, and component-compatibility merge is justified by a relation-matrix rule. When central conflicts occur, close them as a native congruence in A , replace the carrier by the resulting quotient, and restart from the projection of the original observations. These are sound central consequences and implement both local and context feedback.

At a fixed point without central conflicts, expand the implicit cells conceptually. The local congruences, Lemma 4.1, and the quotient representation satisfy every rule of Theorem 3.1. Soundness and leastness therefore identify the result with R^* . At a fixed Q , the number of possible labelled endpoints and equality merges is finite. The central congruence can change strictly at most $n - 1$ times. An uncapped fair monotone implementation consequently terminates; a prematurely exhausted engineering budget must return unresolved. $\square$

Algorithmic order. Start with the diagonal central congruence. Build Q and projected observations; close mirrors and local congruences, then both compatibility cases, repeating until stable or a central conflict is found. If a conflict appears, enlarge θ by its native congruence closure and rebuild over Q . Return only after a stable round with no conflict. This specification exposes the conditions an optimized representation must preserve, rather than assuming that a choice of data structure establishes completeness.

4.1 Principal generators and a width-dependent carrier

For $s \leq u$ in a join-semilattice, the comparably generated principal congruence satisfies

$$\text{Cg}(u, s) = \Delta \cup \{(a, b) : s \leq a, b, u \vee_A a = u \vee_A b\}. \quad (4.1)$$

We use this established characterization [8, Theorem 2.1], together with

$$\text{Cg}(x, y) = \text{Cg}(x \vee_A y, x) \vee \text{Cg}(x \vee_A y, y).$$

Grouping the elements $a \geq s$ by $u \vee_A a$ supplies generators for each class in (4.1). Inserting these pairs directly into a persistent local disjoint-set structure computes its join with the new principal congruence; rebuilding a separate full-carrier structure for each seed is unnecessary.

On nonempty KnownBits values at width w , native join retains the bits known in both arguments:

$$(Z, O) \vee_A (Z', O') = (Z \& Z', O \& O').$$

There are 3^w carrier values. Above a fixed element, the original discrete carrier consists of submasks of its known literals, so at most 2^w values need inspection for a comparable principal generator. After a nontrivial quotient, a general quotient-carrier scan remains a valid option. These observations concern representation cost in a width-dependent finite algebra; they do not give a polynomial-time algorithm in w .

The historical component-pair C++ backend implements these ideas and has substantial differential evidence. Its source includes a 100-pass loop without a distinct exhaustion result. Therefore Theorem 4.2 is a proved representation contract, not an unconditional correctness theorem for that exact historical file. The public Python certifier has its own explicit resource-limit outcomes and does not call this backend.

4.2 Algebraic compatibility does not establish concrete soundness

A constant τ preserves a positive-arity semilattice operation separately in both arguments. Choosing its constant to be a wrong concrete answer gives an algebraically compatible but

unsound transformer. Consequently relation closure cannot turn arbitrary sampled examples into a soundness proof for an unrestricted candidate language.

Nor does every optimal transformer preserve this native join. At width two, let $a = 00$, $a' = 11$, and $b = 01$ denote exact words. Then $a \vee_A a' = ??$, and the best modular-add transformer satisfies

$$\text{Add}^\#(a \vee_A a', b) = ??, \quad \text{Add}^\#(a, b) \vee_A \text{Add}^\#(a', b) = 01 \vee_A 00 = 0?.$$

Thus separate join compatibility is not a general axiom of optimal KnownBits addition. The word-certification route below uses the actual expression semantics instead of importing such an axiom.

5 The complete finite semantics of the accepted word fragment

5.1 KnownBits and the coordinatewise language

For $w \geq 1$, write $B_w = \{0, \dots, 2^w - 1\}$ and let

$$\text{KB}_w = \{(Z, O) \in B_w^2 : Z \& O = 0\}.$$

Its concretization is

$$\gamma_w(Z, O) = \{x \in B_w : x \& Z = 0, x \& O = O\}.$$

Inputs are nonbottom: every bit is known zero, known one, or unknown. We use $\mathbb{K} = \{Z, O, U\}$ for these three one-bit states, encoded respectively by $(1, 0)$, $(0, 1)$, and $(0, 0)$. The context distinguishes the state symbols from word masks. An output with conflicting masks has empty concretization and cannot be sound for the nonempty target images considered here.

The accepted mask language has four variables z_a, o_a, z_b, o_b , the constants zero and all-ones, and bitwise AND, OR, XOR, and NOT. A pair $e = (e_Z, e_O)$ consists of two such expressions. The same syntax is interpreted at every width. The ordinary integer constant 1 is not an all-ones constant at widths above one and is therefore outside this final language, as are counts, shifts, comparisons, and arithmetic residuals.

KnownBits and the closely related tnum domain are established abstract domains [6, 7]. We use raw zero/one masks so that the exact interface matches the program's return value. Our class counts below do not identify distinct invalid masks merely because they denote bottom.

Lemma 5.1 (Coordinate evaluation). *For every accepted mask expression e , positive width w , and bit position $j < w$, its output bit depends only on the four input-mask bits at j and equals the one-bit evaluation of e on those bits.*

Proof. Variables and the two constants have the stated property. Each permitted operation applies its Boolean operation independently at each bit. Structural induction gives the result. $\square$

Theorem 5.2 (Nine-row soundness and optimality, K5.1). *Let f_w be the coordinatewise lift of a Boolean binary target f_1 . An accepted pair expression e is sound at every positive width if and only if, for every $(a, b) \in \mathbb{K}^2$,*

$$\{f_1(x, y) : x \in \gamma_1(a), y \in \gamma_1(b)\} \subseteq \gamma_1(e_1(a, b)). \quad (5.1)$$

Replacing inclusion by equality gives a necessary and sufficient criterion for the best KnownBits result at all positive widths. A failed soundness row yields a concrete width-one counterexample.

Proof. Necessity follows by choosing width one. For sufficiency, every valid word input is a product of its one-bit possibilities. By Lemma 5.1, (5.1) at each coordinate includes the corresponding concrete target bit in the output. This proves soundness of the whole word.

For optimality, coordinatewise target evaluation on product inputs has a product image: choices of the input bits at different positions are independent. Each coordinate image is a nonempty subset of $\{0, 1\}$ and is represented exactly by one of Z, O, U . Equality in all rows therefore gives exactly the product image, which is the best domain value. If inclusion fails, the offending row contains concrete bits whose target result violates an output mask, already at width one. $\square$

For the three supported targets the exact table is:

a	b	AND	OR	XOR
Z	Z	Z	Z	Z
Z	O	Z	O	O
Z	U	Z	U	U
O	Z	Z	O	O
O	O	O	O	Z
O	U	U	O	U
U	Z	Z	U	U
U	O	U	O	U
U	U	U	U	U

5.2 An extensional quotient, not a syntactic normal form

Order the input rows lexicographically using (Z, O, U) on each argument, with the second argument varying fastest. Encode the zero and one output bits of row i in positions $2i$ and $2i + 1$:

$$\sigma(e) = \sum_{i=0}^8 \left(e_Z(i)2^{2i} + e_O(i)2^{2i+1} \right).$$

This is the exact 18-bit signature reported by `inspect`.

Theorem 5.3 (Complete finite signature, K5.2). *Two accepted pair expressions have equal signatures exactly when they return identical masks at every positive width on every valid input. There are exactly 2^{18} raw-mask semantic classes and 3^9 classes with valid outputs on every valid input. Evaluation of one mask is a Boolean-algebra homomorphism onto the 512-element algebra of Boolean functions on the nine rows.*

Proof. Equality of signatures is equality on the nine rows and lifts coordinatewise by Lemma 5.1; conversely, take width one. To show surjectivity, each valid assignment of the four mask bits has a Boolean minterm using variables and their negations that singles it out among the nine rows. Disjunctions of these minterms define every subset of rows. Thus one mask has 2^9 classes, and two independently chosen masks have 2^{18} . Restricting each row to the three nonconflicting outputs gives 3^9 . Boolean operations are evaluated pointwise, proving the homomorphism assertion. $\square$

The kernel of this evaluation is an exact semantic congruence. It is a second concrete instance of the evaluation-kernel viewpoint from Proposition 2.1, but its semantic models and permitted expressions are specified independently of E_τ . The implemented normalizer is not claimed to choose one unique syntax for each signature.

For componentwise mask meet, $(Z, O) \sqcap (Z', O') = (Z \mid Z', O \mid O')$, the signature obeys

$$\sigma(e \sqcap e') = \sigma(e) \text{ OR } \sigma(e'). \quad (5.2)$$

On arbitrary raw outputs this operation can produce conflicting masks. On sound transformers of the same target, the common nonempty concrete image prevents that conflict.

5.3 The exact precision classes

Let $\mathfrak{F}_f$ be the input rows on which the target image is a singleton, annotated by that forced bit. These are the possible correct precision facts. No monotonicity assumption is imposed in the following classification.

Theorem 5.4 (Sound precision lattice, K5.3). *Sound coordinatewise transformer classes for f are in bijection with subsets of $\mathfrak{F}_f$. Every subset is expressible in the accepted language. Meet corresponds to union of precision facts. For AND, OR, and XOR there are respectively 64, 64, and 16 sound classes, and a chain of strict meet refinements starting at all-unknown has at most 6, 6, and 4 strict steps.*

Proof. When the target row image is $\{0, 1\}$, soundness forces output U . When the image is a singleton, the only sound outputs are its correct singleton and U . Thus a sound table chooses exactly which forced facts to retain. Theorem 5.3 gives definability of every such table, and (5.2) gives union. The displayed target table has six, six, and four forced rows. Each strict refinement adds at least one previously absent fact. $\square$

These heights bound semantic precision within the accepted fragment. They do not bound candidate-generation iterations, proof-search cost, or the cost of the best implementation of a class.

Proposition 5.5 (Additional monotonicity). *If transformers must also be monotone for the precision order induced by concretization inclusion, the numbers of sound classes are 26, 26, and 16 for AND, OR, and XOR.*

Proof. A retained forced fact at an input row requires the same fact at every more precise row. For AND, the five zero rows have three minimal rows (Z, Z) , (Z, O) , (O, Z) and two further rows (Z, U) , (U, Z) . Selecting (Z, U) requires the first two minimal rows; selecting (U, Z) requires the first and third. The number of downsets is $8 + 2 + 2 + 1 = 13$, according to whether neither, only the first, only the second, or both further rows are selected. The one row (O, O) forcing one is independently optional, giving 26. OR is dual. All four forced XOR rows are minimal, so every subset is monotone. $\square$

The release computes the 18-bit signature and target soundness/optimalty; it does not expose a separate monotonicity flag. The additional classification is a mathematical result checked in the supplement and a possible future interface extension.

6 From source text to a checked all-width verdict

6.1 The source-semantics contract

The source bundle maps caller-chosen labels to UTF-8 program texts. The entry function takes two KnownBits pairs. The supported grammar is a restricted, line-oriented subset of transfer MLIR: single-block typed SSA functions, pure operations, acyclic calls with explicitly supplied definitions, and an actual return instruction. It is not the full MLIR language. Unknown

syntax, unresolved calls, recursion, type inconsistencies, and exhausted resource budgets do not produce a successful target verdict.

At width $w \geq 1$, word arithmetic is modulo 2^w . Boolean expressions have separate two-valued semantics. Counts return integers in $[0, w]$, encoded as words; $w < 2^w$ ensures this is possible. Shifts and division use the total semantics specified in Appendix A. In particular, the source operation `transfer.neg` denotes bitwise NOT in this contract. No poison or undefined behavior is introduced. Since every operation is pure and total, discarding an unused computation preserves the returned value.

Lemma 6.1 (Expression construction). *For this grammar and semantics, symbolic evaluation of SSA instructions, expansion of acyclic calls, and projection of the actual return produce an expression e_0 with the same value as the source entry at every positive width on every valid input.*

Proof. Maintain an environment mapping each defined SSA name to an expression with its current semantic value. Input projections initialize the four mask variables. Pair construction and projection preserve the pair semantics. Each primitive instruction adds its specified operation to already-correct operand expressions. A call substitutes the actual argument expressions for the callee’s formal arguments; induction on the acyclic call graph justifies its expansion. Induction on instruction order maintains the environment invariant. Selecting the expression for the actual return gives the result. Totality justifies ignoring expressions that do not contribute to that return. $\square$

The frontend module performs this translation through its `parse_bundle` and `expression` functions. The lemma describes the translation algorithm; correspondence of the Python parser with the documented text grammar is part of the implementation trust described below.

6.2 Local rules and semantic guards

The release has 29 rule identifiers. Most implement direct constant, idempotence, cancellation, count, comparison, and selection laws. Two guards reason about support: `disjoint` establishes that two operands have no common one bits, and `subset` establishes inclusion of one operand’s support in another’s. On coordinatewise expressions they exhaust the nine valid input rows; structural cases use AND restriction and OR inclusion. The `lowbit` guard proves a constant low bit using bitwise laws and parity of modular addition/subtraction.

Lemma 6.2 (Sound guards). *If the release’s support guards return true, their respective disjointness or inclusion assertions hold at every positive width on every valid input. If `lowbit` returns $c \in \{0, 1\}$, the operand has low bit c at every positive width.*

Proof. For coordinatewise guards, Lemma 5.1 lifts the exhaustive one-bit checks. The structural disjointness cases replace an AND-expression by an operand with a superset of its support, so established disjointness is preserved. The subset cases use reflexivity, empty support, full support, inclusion into either branch of OR, and inclusion from either operand of AND.

For the low bit, zero, all-ones, and integer literals have their stated parity. A shift left by the word constant one clears that bit, including at $w = 1$. NOT flips it, AND/OR use their Boolean laws, and XOR, modular addition, and modular subtraction have low bit equal to parity XOR. Each recursive case is therefore sound. $\square$

For example, if $x \& y = 0$, addition has no carries and $x + y = x | y$ at every width. If $\text{supp}(x) \subseteq \text{supp}(y)$, setting the leading `countl_one(x)` bits of y changes nothing. These are precisely the guarded rules `disjoint-add` and `set-leading-included`. Appendix A gives the complete rule catalogue and proof arguments for the remaining families.

Proposition 6.3 (Local rewrite preservation). *Every applicable rewrite in the mathematical rule system listed in Appendix A preserves value at every positive width on valid KnownBits inputs.*

Proof. The guarded rules use Lemma 6.2. The remaining rules follow from the stated modular, Boolean, order, count, and total division/shift semantics, as checked by the cases in the appendix. In particular, the rules involving zero remainders remain valid at divisor zero under this contract. Replacing an equal-valued subexpression preserves every enclosing operation by congruence. $\square$

6.3 Termination without a normal-form optimality claim

For an expression tree e , let $N(e)$ count operation and leaf nodes, with numeric literal payloads treated as part of a leaf, and let $A(e)$ count occurrences of word addition. Shared occurrences are counted as occurrences in the expanded tree. Put

$$\mu(e) = N(e) + A(e).$$

Proposition 6.4 (A decreasing measure). *Every rewrite step of the release decreases μ strictly. Thus any rewrite sequence has at most $\mu(e_0) - \mu(e_N)$ steps. In particular the unbudgeted normalization strategy terminates.*

Proof. The rule `disjoint-add` changes an addition node to an OR node without changing its children; it preserves N and decreases A by one. Every other rule replaces a term by a proper subterm or a single constant, hence strictly decreases N and introduces no addition. In an enclosing context the change in μ is unchanged. The nonnegative integer measure proves termination and the bound. $\square$

This statement does not claim confluence, a unique syntactic normal form, minimum proof length, or minimum depth. The producer uses a fixed rule order after recursively processing children. Its finite proof-step budget can stop a terminating computation early; the API then returns `fallback_required`. The extensional signature of Section 5, when applicable, provides canonical semantic information independently of syntactic choices.

6.4 Replay and the composition theorem

A certificate records the schema identifier, source-label fingerprints, entry, semantic contract, initial-expression fingerprint, a sequence of rule/path instructions, and a claimed final expression. For a target certificate it also records the nine rows and target name. The caller supplies the source bundle and target again. The checker reparses that source, checks the bindings, replays every instruction, compares the obtained expression with the claimed one, and recomputes the target rows. A certificate cannot introduce an unproved assumption or substitute its own target for the caller’s target.

Theorem 6.5 (Source-to-verdict composition, K4). *Assume the frontend implements the source contract of Lemma 6.1, the checker implements the rules of Proposition 6.3, and replay successfully establishes $e_0 \rightsquigarrow e_N$. Then the source and e_N return the same value at every positive width on valid inputs. If e_N is an accepted pair and its recomputed table for the external target passes, the source transformer is sound at every positive width. If every table row is optimal, so is the source transformer. A failed row supplies a width-one refutation of the source.*

Proof. Induct on the replay instructions. Each instruction selects a valid subtree path and replaces its current subexpression by an instance of a value-preserving rule, establishing equivalence of successive whole expressions. Transitivity and Lemma 6.1 establish source equivalence with e_N . Apply Theorem 5.2 to the recomputed rows. Its concrete witness also refutes the source because normalization preserved the returned masks. $\square$

The producer’s search strategy is absent from this implication. The implementation’s checker does not run that search, although producer and checker share local-rule code. The trusted implementation comprises parsing, call expansion, schema validation, guards, local rules, replay, and the finite table evaluator. The mathematical proofs and finite regressions have not been replaced by a proof-assistant verification of that Python code. Nor does this theorem certify a separate compiler lowering of the source language.

6.5 Result semantics

Result	Meaning under the declared contract
<code>certified</code>	Target soundness at all positive widths; <code>optimal</code> reports the separate precision check.
<code>unsound</code>	Replayed normalization and a concrete width-one target counterexample.
<code>normalized</code>	Source-to-expression equality only; no target soundness claim.
<code>fallback_required</code>	Unsupported source/target/residual or exhausted budget; no complete target verdict.
<code>invalid_certificate</code>	A supplied certificate fails validation, binding, or replay.

Input and internal errors are separate failure outcomes. A normalization-only certificate has a null target and no rows; a target check does not accept it as a soundness certificate. Consumers must accept a transformer only from a successful target verification or check with status `certified`.

7 From consumer observations to whole-word reconstruction

7.1 The observation required by a consumer

Let D be the domain satisfying the checked path conditions of a source expression, and let $C : D \rightarrow Y$ denote its actual downstream consumer. An observation $\pi : D \rightarrow O$ is sufficient when

$$\ker \pi \subseteq \ker C. \quad (7.1)$$

The domain is part of the statement. A condition discovered in a branch cannot be used outside that branch without a new proof.

Proposition 7.1 (Consumer factorization, K6.1). *Condition (7.1) holds if and only if there is a unique map $\overline{C} : \pi(D) \rightarrow Y$ such that $C = \overline{C} \circ \pi$.*

Proof. Define $\overline{C}(\pi(x)) = C(x)$. Kernel inclusion is exactly the condition that this definition be independent of the representative. The equation gives existence, and surjectivity onto $\pi(D)$ gives uniqueness. Conversely, equality of observations implies equality after composition with $\overline{C}$. $\square$

This factorization criterion makes the implementation obligation explicit. An observation must be tied to the source value, its domain, and its consumer; a small observation type alone

proves nothing. Research producers propose queries from actual uses of a result. Checkers reconstruct the source expression and path facts, validate the observation derivation, and replay its substitution at the designated occurrence.

A shift amount represented by modular word arithmetic need not equal the mathematical integer suggested by its syntax. Its action on a particular operand can nevertheless be determined by a small quotient of the amount. A comparison may need only a result sign under a checked no-overflow condition. The implemented families cover particular count, shift, support, sign, and result observations; a complete synthesis procedure for the best observation of an arbitrary program remains open.

7.2 A kernel does not label its classes

Recovering an entire word requires more than proving that an action forgets certain bits. For fixed $0 \leq s \leq w$, put

$$M_s = 2^{w-s} - 1, \quad \pi_s(x) = x \& M_s.$$

The kernel of left shift by s is the kernel of π_s . Every class has one representative among the submasks of M_s . Identifying a proposed inverse with that representative still requires a native semantic equation.

Proposition 7.2 (Labeled shift reconstruction, K6.2). *Under the declared total word semantics, for $0 \leq s \leq w$,*

$$\text{LSHR}_s(\text{SHL}_s(x)) = x \& M_s.$$

Consequently, if $x \& M_s = x$, this round trip returns x . The amount s is fixed in the original environment throughout both actions.

Proof. The left shift discards exactly the s highest bits and moves every remaining bit s positions upward. Logical right shift moves those bits back and fills the highest positions with zero. This is the stated projection; at $s = w$ the retained support is empty. The guarded conclusion follows by substitution. $\square$

The research artifact also derives the universal equation in its preceding symbolic proof system and records its instantiation, support premise, ground equality steps, and consumer substitution. The native projection law is an explicit proof dependency.

The distinction matters on a three-bit carrier. The maps $x \mapsto x \& 3$ and $x \mapsto \text{swap}_{0,1}(x \& 3)$ have the same kernel and agree at 0, 4, 3, but disagree at 1. Kernel information and those supplied values do not identify the internal labels. This is a countermodel to an insufficient interface, not an execution of the actual shift round trip.

7.3 Rows forced by atomic observations

For a finite observation set Q with at least two elements, use the named carrier $A = \mathcal{P}(Q)$ with union and intersection. A native cell action determines a map $\phi : Q \rightarrow Q$ and a guard $G \subseteq Q$. Its backward observation row is

$$h(P) = G \cap \phi^{-1}(P). \tag{7.2}$$

Lemma 7.3 (Atomic completion, K7.1). *The row in (7.2) preserves union and intersection. Its values on singleton subsets determine its complete table among all maps preserving these operations.*

Proof. Membership in either side of each preservation equation is equivalent to membership in G and the corresponding Boolean combination of $\phi(q) \in P$. Singletons generate nonempty subsets by unions; the intersection of two distinct singletons generates the empty set. Induction on these expressions proves uniqueness from the supplied atomic values. $\square$

This is a finite instance of the forced-row viewpoint of Paper I. In the examples below the supplied atoms generate the whole carrier: $S = D = A$, and the row kernel is $\kappa = K = \ker h$. A total realization on the same named carrier witnesses protection of its distinct central elements. Row kernels can be nontrivial while the central relation remains diagonal. No growth of the forced domain beyond the generated subalgebra is claimed here.

The completed tables are consumed by the transitions of the checker. The ground tables describe the named carrier; they do not assert lattice identities on every unnamed element of an arbitrary total model.

7.4 Finite gluing and its semantic bridge

Let $\delta : S \times B \rightarrow S$ act on observations and bit-column types, with initial observation s_0 and acceptable final observations $\text{Good} \subseteq S$. A certificate supplies a finite carrier and an edge table over every legal column.

Theorem 7.4 (Closed observation carrier, K7.2). *If the supplied carrier contains s_0 , is closed under every $\delta(\cdot, b)$, and every one of its states satisfies **Good**, then processing any finite list of columns from s_0 ends in **Good**. More generally, a relation between a concrete partial-word state and its observation that holds initially and is preserved by every local step holds after every finite list.*

Proof. Induction on the column list preserves membership in the closed carrier; its final-state condition gives the first assertion. The same induction preserves the simulation relation and gives the second. $\square$

Unreachable certificate states are harmless if they also satisfy the obligations. Minimality of the state count is unnecessary. A word theorem additionally needs completeness of the column alphabet, a bridge from comparisons to numerical word values, and a sign-boundary argument. A graph check without these bridges establishes only a statement about that graph.

The procedure identifies the consumer and domain, derives labeled native observations, completes compatible rows, searches for a closed carrier, checks its local and boundary obligations, and lifts the result through the semantic relation. Source binding and replay remain necessary at each substitution. It does not assume that every optimal abstract transformer preserves KnownBits join.

8 Signed words and joint mask–interval carriers

8.1 The carrier that must be preserved

For width $w \geq 1$, let m and a be compatible must/may masks, $m \subseteq a$, and let sgn_w interpret a raw word as a signed integer. Define

$$\mathcal{M}(m, a) = \{\text{sgn}_w(x) : x \& m = m, x \& \neg a = 0\}.$$

The joint carrier also has signed bounds and a zero flag:

$$J(m, a, L, U, Z) = \{x \in \mathcal{M}(m, a) : L \leq x \leq U, Z \vee x \neq 0\}. \quad (8.1)$$

An interval between mask extrema generally contains words outside the mask. Preserving those extrema alone does not establish preservation of (8.1). The Graal study keeps mask membership and order facts as distinct proof information.

The source is `IntegerStamp.java` [11], revision

3f5efeb49d934f8a0bcb3c115a6a28caa5622975.

The restricted source bridge checks actual helper bodies, loop directions, branches, guards, and returns. Supporting mask extrema and Java operations have declared native semantics and finite differential checks. This bridge has not been verified as a general Java compiler.

8.2 Descending repair needs a common higher context

Within one permitted sign half, let b be the minimum masked word. The descending pass visits optional nonsign positions from high to low and sets a position precisely when the trial word is at most U . At position i that word is

$$y_{>i} \ 1 \ b_{<i}.$$

The higher prefix is shared by every decision. Permitting a separate higher-prefix witness at each position admits combinations that no execution can produce.

The observation retains $P \subseteq R$, where $R = \{-1, 0, 1\}$ describes possible relations of the common higher prefix to the higher prefix of U . Write $p \triangleright q = p$ when $p \neq 0$, and q otherwise. For a cell, with lower comparison $r = \text{cmp}(b_{<i}, U_{<i})$, put

$$\phi(h) = h \triangleright \text{cmp}(y_i, U_i), \quad t(h) = h \triangleright \text{cmp}(1, U_i) \triangleright r.$$

The source predicate gives the guard G of higher relations selecting the proposed output bit. Updating P is exactly (7.2). The certificate assembles constraints upward while retaining unresolved higher context, describing the decisions of the downward execution.

The carrier $\mathcal{P}(R)$ has eight elements. Three supplied atoms force the other five cells of each of 24 row templates. The row kernels have two, four, or eight classes. The full observation is

$$(P, \text{cmp}(b, U), \text{cmp}(x, U), \text{cmp}(y, U), \text{cmp}(x, y)),$$

where comparisons refer to processed lower parts. Of at most $8 \cdot 3^4$ observations, the certificate supplies 49 states, 588 transitions over 12 legal column types, and 196 sign completions.

Proposition 8.1 (Descending masked maximum). *In the selected sign half, if a masked word $x \leq U$ exists, the descending pass returns a masked y with $y \leq U$ and $x \leq y$ for every such x .*

Proof. The initial word is the minimum of the sign half. Before each choice its unvisited optional bits are zero. Setting the current bit is feasible exactly when the trial word, the minimum completion of that proposed prefix, is at most U . If feasible, setting it maximizes the prefix; otherwise no admissible word with that prefix can set it. Induction over descending positions proves the result. The observation certificate records those same decisions through the shared P , and its complete transition and signed-end checks lift by Theorem 7.4. $\square$

Forgetting P is insufficient already at width two: the weakened interface permits output 1 for mask carrier $\{0, 1\}$ and $U = 0$. This separates that weakened interface from the required one; it does not establish minimality of all 49 states.

8.3 The complete upper helper

Put $C = \{x \in \mathcal{M}(m, a) : x \leq U, Z \vee x \neq 0\}$.

Theorem 8.2 (Upper contract, K8). *Under compatible masks, a typed signed upper bound, and the declared native source bridge, the modeled `computeUpperBound` returns $\max C$ when C is nonempty. Its empty branch is reached exactly when C is empty. Replacing U by the actual returned upper value preserves (8.1) for every additional lower bound L .*

Proof. Proposition 8.1 handles each invoked pass. The outer control selects sign halves and handles forbidden zero. Its comparisons involve seven named values: zero, relevant mask extrema, U , a pass result, and an arbitrary witness x . Every execution maps to their full weak order. All 47,293 ordered partitions of seven names are enumerated; the source IR is checked on the 744 cases consistent with necessary mask and pass facts. Membership is tracked by the provenance of returned values, in addition to their order.

In every admitted case a value return belongs to C , and any $x \in C$ forces that branch with $x \leq \text{result}$. These assertions establish maximality and characterize the empty branch. A nonempty maximum removes no word from an additional lower-bounded intersection. In the empty case the minimum-word return is at most the original typed U , so the tightened intersection stays empty. $\square$

The numerical minimum word can also be a genuine maximum. The extra empty-case observation distinguishes these situations; numerical equality to a sentinel is insufficient.

8.4 Ascending repair and the carry quotient

The lower helper first uses a descending pass. If its result g is below L , an ascending repair seeks the next masked word. Before a nonsign cell there are three reachable physical phases:

Phase	Increment started	Physical carry	Observed carry q
A	no	0	1
B	yes	0	0
C	yes	1	1

Before incrementing, $q = 1$ means a pending successor request; afterwards it means an unconsumed carry. Source-cell evaluation shows that A and C have identical output bits and next observation classes on every legal cell. Phase B is separated by an optional zero input.

A fixed bit remains fixed and passes q through. For an optional bit the source-derived action is

$$(g_i, q) \mapsto (g_i \text{ XOR } q, g_i \& q).$$

For each cell label and proposed output y , the backward row is

$$h_y(P) = \{q : \text{out}(q) = y, \text{next}(q) \in P\}.$$

Eight such rows act on $\mathcal{P}(\{0, 1\})$. Their two supplied atomic values force the complete four-cell tables, from which the transition reconstructs its output and next carry.

The observation records q , comparisons $g/x, t/x, g/t$, two extremal-mask flags, and whether an optional bit occurred. Its closed carrier has eight states, 48 transitions over the six legal columns (m_i, a_i, g_i, x_i) , and 32 permitted sign completions.

Theorem 8.3 (Masked successor). *For any positive width, if an admissible masked word $x > g$ exists, the mathematical successor construction returns a masked word t satisfying*

$$g < t \leq x.$$

No positive-to-negative overflow occurs in that situation.

Proof. The source-cell factor supplies the local action, and the forced rows recover it. The certificate contains the empty-suffix observation and is closed under every legal nonsign column. Its sign checks establish the comparison and mask obligations. Comparison of lower parts is updated by letting a newly added higher bit take priority; the final signed bit reverses the order of its two values. These numerical relations and Theorem 7.4 prove the result for every list length, including an empty nonsign list. $\square$

Without nonsign optional bits, the virtual successor observation is still defined. The actual helper has a separate branch returning an intermediate zero. Relating the executed ascending loop to the sign carry uses the presence of an optional bit; the virtual convention is not substituted for that outer branch.

8.5 The lower helper has a conditional preservation contract

Theorem 8.4 (Lower contract, K9). *Assume compatible masks and typed signed L , and additionally*

$$L \leq 0 \quad \text{or} \quad a_{w-1} = 0. \quad (8.2)$$

Under the declared source bridge, the lower-helper result R satisfies $R \geq L$ and

$$x \in \mathcal{M}(m, a), \quad x \geq L, \quad (Z \vee x \neq 0) \implies R \leq x.$$

Thus replacing L by R preserves (8.1) for any U .

Proof. Fix such a witness x . If the minimum masked word already suffices, the corresponding source branch is safe. Otherwise the first pass gives the largest $g \leq L$ in its sign half. When $g < L$ and nonsign optional bits exist, Theorem 8.3 gives $g < t \leq x$. A successor in the same sign half cannot satisfy $t \leq L$, since that contradicts maximality of g . A change to the nonnegative half gives $t \geq 0 \geq L$ by (8.2).

Without nonsign optional bits, each permitted sign half is a singleton. A witness larger than the initial negative singleton forces a nonnegative half and makes the intermediate zero safe under (8.2). If zero is forbidden, a positive witness permits the source's least-positive masked correction: either the nonzero must mask or the least permitted power of two, no larger than x . The final guard ensures $R \geq L$, including cases without a witness.

The checker verifies the complete source control IR over seven named values. Their 47,293 weak orders and two Boolean flags give 189,172 combinations; 25,737 satisfy the necessary facts. Unused intermediate values stay unconstrained. Every real execution with a witness is therefore represented, and all admitted executions satisfy $L \leq R \leq x$. These inequalities give joint-carrier preservation. $\square$

The result need not be an exact minimum. At width eight, $m = 1$, $a = 129$, $L = -1$, and permitted zero, the helper returns 0, while the least feasible masked word is 1. Conversely, with $m = 0$, $a = 192$, and $L = 1$, it returns 127 although 64 is feasible. This second input violates (8.2); it explains why that condition cannot be dropped from the current contract. Neither helper example is a counterexample to the complete Graal constructor.

For an empty carrier, the contract imposes neither mask membership nor w -bit representability on the returned lower value. Consumers must process the empty outcome before reusing such a value as a typed bound. Full `create` also updates temporary masks, passes through potentially incompatible states, and limits stabilization to three iterations. Its complete preservation and termination argument is the next composition task.

9 The first Lean verification

The pilot is a standalone Lean 4.33.0 project using the standard library without Mathlib [12, 13]. It formalizes the ascending mathematical fragment above. Certificate constants are exported from the saved research certificate as data; handwritten definitions and general lemmas determine what those constants must prove.

Module	Checked content
RowRules	Union/intersection on $\mathcal{P}(\{0, 1\})$, inductive forced-cell rules, soundness, atomic uniqueness, and row-kernel compatibility.
Gluing	Simulation and invariant preservation along every finite column list; soundness of a closed certificate.
Core, Data	Source-cell phases and their factor; the complete six-column alphabet; row, state, and edge tables.
Checked	Atomic agreement, row uniqueness, recovery of the native action from completed rows, closure, and sign conditions.
WordSemantics	Numerical words, lower-part bounds, signed comparisons, output-mask membership, and the all-width successor theorem.
SourceBridge	Equality of observed numerical states and output traces after any number of modeled nonsign source cells.
Audit	Expected axiom dependencies of the principal theorems, enforced at build time.

Theorem 9.1 (Lean pilot scope, K10). *The distributed Lean source proves the masked-successor result for every list of legal nonsign columns and each allowed sign completion satisfying its witness premise. It proves the source-word factor for the explicitly modeled nonsign repair iteration. Their names are*

QKF.mask_successor_all_widths
QKF.source_word_factor.

The length lemma identifies the complete signed width as the list length plus one.

Proof. The checked certificate supplies the initial state, every transition, and every sign obligation. General gluing yields the invariant for arbitrary lists. A separate relation connects observations to accumulated natural-number values and proves each lower part less than its power-of-two bound. Signed comparison translates the relational assertions into $g < t \leq x$, while trace induction establishes mask membership. For the source bridge, local phase factorization commutes with each numerical step and is lifted by the simulation theorem. These are the dependencies of the named Lean proofs. $\square$

Finite obligations use `decide`; elementary integer bridges use `omega`. The audit reports no axioms for forced-rule soundness and atomic row uniqueness. The main numerical theorem uses only the standard `propext` and `Quot.sound`; the source-word factor uses `propext`. No additional axioms, proof placeholders, `native_decide`, or `bv_decide` are used. Exact expected axiom lists are enforced by `#guard_msgs`.

The pilot does not formalize both foundational papers in full, the 49-state descending certificate, the outer lower-order argument, the complete Java helper, or the Python checker. Its source bridge concerns the stated operational cell model; translating actual Java and host-long arithmetic remains a separate obligation. Lean verification improves confidence in this fragment without changing whole-program coverage counts.

The package has been built from a fresh extracted copy, and three numerical examples are checked. Four isolated negative controls change an initial state, an edge, a row cell, and a proof; all are rejected at the corresponding certificate or axiom-audit gate. These controls exercise the build and trust checks; proof terms establish the general results.

10 Implementation profiles and claim traceability

10.1 Coordinatewise CLI

The installable Python package exposes inspection, normalization, target verification, and certificate checking. Its runtime needs only the standard library. Verification constructs a source expression and produces a rewrite trace. Checking reparses the source and replays the supplied trace without invoking normalization search. The two paths share primitive-rule code.

The certificate format remains `qkf-rewrite-v3`, API results use `qkf-result-v1`, and the semantic contract is

```
total-transfer-words-v2; positive-width;  
modular-arithmetic; SMT-total-div-shift;  
standard-counts; nonbottom-KB.
```

The accepted final coordinatewise targets remain AND, OR, and XOR. A resource limit or unsupported residual produces an explicit unresolved outcome. SHA-256 fingerprints bind the complete supplied source-label map, including helper definitions; they identify bytes, not authorship.

The original three Nice to Meet You examples retain their upstream provenance and MIT notices. AND normalizes to $(z_a \mid z_b, o_a \& o_b)$; OR has the dual optimal table. XOR expands the actual meet helper and uses a checked disjoint-support addition to recover its optimal table. The checker does not assign semantics from the name of a helper.

10.2 Research capsules

The release distributes the broader research under `research/knownbits` and `research/graal`, with its original code and source identities. The KnownBits capsule preserves the result-observation stage and its packed experiment data. The Graal capsule preserves the universal lower stage together with its upper and joint-carrier dependencies. A release runner restores checked data and starts fresh replay processes; existing experiment outputs are retained.

These profiles require Python 3.12 on Linux/POSIX for their existing signal and resource controls. Replaying saved proofs does not need SMT, Java, or the producers. Regenerating native Java comparisons requires the separately documented JDK and compiler. Actual Java execution is a finite differential check of extracted helper bodies and shims; the whole Graal build is not part of it.

The checked upper and lower APIs accept concrete mask–interval descriptions after validating the corresponding universal contract. Their operational width limit is 4096 bits. This limit bounds an implementation request and is absent from the mathematical induction theorem. The lower API explicitly rejects source sign contexts outside (8.2).

10.3 Lean profile and stable claims

The project in `research/lean` is checked with `lake build`; its toolchain is pinned separately from Python. It includes its data exporter, axiom audit, examples, negative controls, and actual build logs. Table 1 identifies the principal claims.

The baseline parser, kernel, producer, and replay modules can still be inspected at the immutable public commit given in Section 1. New files are identified by the release manifest. Historical measurements and the newly executed release checks have separate records. No experiment is promoted to a larger source or semantic scope merely by being bundled in this release.

Claim	Artifact	Scope
K1–K2	Paper supplement: finite core and relation-matrix oracle	Exact finite ground interface; independent finite checks.
K3	Sparse prototype and representation theorem	Conditional coverage and fixed-point contract; historical pass-limit caveat retained.
K4–K5	<code>src/qkf_certifier</code>	Coordinatewise source replay and complete nine-row semantics.
K6	Research observation kernels and guarded native bridges	Consumer-specific domains and explicitly labeled actions.
K7	Graal row/closure kernels; Lean <code>RowRules</code> , <code>Gluing</code>	Atomic completion and finite closure lifted to arbitrary lengths.
K8	<code>research/graal/previous</code>	Universal upper model and limited source bridge; not Lean-checked.
K9	<code>research/graal</code>	Conditional lower preservation; not an exact-minimum or full- <code>create</code> theorem.
K10	<code>research/lean/QKF</code>	Machine-checked ascending mathematical fragment and nonsign source-cell model.

Table 1: Claim identifiers remain stable while theorem numbers are local to this revision.

11 Evidence, counterexamples, and remaining coverage

11.1 Retained baseline evidence

Manuscript v1.0 audited the public 0.1.0a1 commit on Python 3.12.14 and x86-64 Linux. Its retained evidence includes 67 passing tests, 22,140 concrete comparisons of the original AND/OR/XOR programs at widths one through four, and an independent matrix-versus-ground comparison on 804 instances and 35,856 interface pairs. All 29 rewrite identifiers had witnesses, with 23,751 finite before/after comparisons and checks of the decreasing measure.

The finite classification checked all 512 Boolean row functions and all 3^9 valid output tables per target, confirming sound-class counts 64, 64, 16 and monotone counts 26, 26, 16. Of 680 source mutations, 270 were certified, 180 refuted at width one, and 230 required fallback. These remain evidence about the pinned coordinatewise profile. Versioned logs retain the original timings; this revision does not treat them as new performance measurements.

11.2 Broader research has several denominators

Dataset or profile	Result	Interpretation
Known LLVM transcriptions	11/11	A development/regression set, distinct from the original 39-program corpus.
GCC 15.2 specializations	9/9	Frozen external transfer study over the selected nine dispatcher specializations.
Original Nice to Meet You solutions	11/39	Complete source-bound proofs in the declared research contract.
Their individual components	104/411	Component proofs; not additional complete programs.
Five Graal procedures, control	1/5	Complete mask result for OR.
Five Graal procedures, joint	2/5	Complete mask results for AND and OR.

Table 2: These populations and proof obligations are not interchangeable.

On the original 39 solutions, the result-observation stage increased the preceding control from 9/39 complete programs and 87/411 components to 11/39 and 104/411. Add reached

13/13 components and UsubSat 16/16. All 411 source components were prepared; source-to-expression preparation alone is not target verification. The other 28 programs include 23 targets without the required research target adapter and five flag-related strengthened obligations. Refutations of those unconditional strengthened obligations do not establish errors in the upstream guarded contracts.

The historical GCC study froze its backend and selection before outcomes. It certified nine SSA specializations from GCC 15.2 and replayed them in separate processes. Composition reduced final certificate states from 2344 to 1070 at unchanged coverage. Accounting for all search attempts gives 2344 versus 1474 states; the smaller certificate count is not the full search cost. Z3 also solved all 72 selected finite-width obligations. These are transfer and structural results on that selected corpus.

11.3 Context and reconstruction evidence

The whole-word reconstruction study retained all claims of a normalization query while replacing a 4401-node subproof by 26 recorded steps, including the universal projection lemma, applicability conditions, ground steps, and substitution. The complete proof decreased from 4905 to 530 steps. On its 18 constructed evaluation sources, the new mechanism proved 11 intended positive cases, left two positive cases unsupported, and returned unresolved on five faulty cases for which separate finite counterexamples were retained. It was a constructed evaluation, not a blind external benchmark.

The evidence also preserves cases where observation machinery increased cost without reducing a proof. The three-bit equal-kernel/different-label separator in Section 7 and the two-bit shared-prefix separator in Section 8 identify concrete information that cannot be discarded. Neither is an upstream program bug.

11.4 Upper and lower contract checks

Check	Recorded result
Upper, all masks/bounds/zero flags at widths 1–6	111,972 cases agree with exact masked maximum and emptiness.
Upper, extracted Java helper	114,040 calls agree with the declared model.
Upper, additional lower bounds at widths 1–4	24,174 joint carriers preserved exactly.
Lower, all small typed cases at widths 1–6	111,972 cases: 76,104 admitted by the sign contract; 35,868 explicitly unsupported.
Lower, extracted Java helper	114,044 calls agree with the declared model.
Lean pilot	Clean build, fresh extraction build, axiom audit, three examples, and four rejected isolated corruptions.

The universal upper certificate additionally has wide mathematical specializations, including 128-, 256-, and 4096-bit cases. These are not Java runs at those widths. Upper and lower tables each describe their own experiment; their related input populations are not added together to inflate coverage. Among upper returns numerically equal to the minimum word, the record separates 37,049 empty cases from 963 genuine maxima.

The lower study retains conservative and out-of-contract behavior, including the examples following Theorem 8.4. A strict-comparison source variant was rejected by the reused floor lemma even though its 2232 checked native cases did not exhibit a bug. Such rejection marks the boundary of a proof interface; it is not by itself an unsoundness verdict.

11.5 What the SMT comparisons establish

One historical comparison used the same stored source-derived formulas for 39 programs at widths 32, 64, and 128, with three solvers and two repeats: 117 obligations and 702 executions. Its unconditional treatment of flag-related programs was later separated from artifact and LLVM-compatible assumptions. A subsequent 32-bit study varied both that contract and the encoding of counts, with a one-second solver budget.

The latter change was substantial. Under the artifact contract, balanced encoding gave 19 UNSAT results for Z3 and 18 for Bitwuzla out of 39, with no SAT results; their prefix encodings gave 13 each. Under the historical contract, balanced encoding gave 16 and 20 SAT results respectively. These runs show why source semantics and encoding must be held explicit before a ranking is meaningful. Their historical versions, budgets, and records remain attached to the reports.

QKF supplies parametric proofs for supported interfaces; these SMT runs ask separate fixed-width questions and did not retain solver proof objects. A useful local result was an all-width QKF proof for Smax when the six recorded 128-bit SMT attempts reached their limits, while at 32 bits conventional solvers could be faster. The evidence supports further controlled comparison and identifies specific successful interfaces. It does not establish a general performance ordering or coverage of arbitrary SMT problems.

11.6 Release verification

The current distribution adds fresh checks of the Python package, restored research replay, Lean build, manuscript references, source identities, and installable distributions. Exact commands, versions, outcomes, and logs are retained in the release validation record. Historical search and native-execution measurements above are not silently rerun or relabeled as this release’s measurements.

12 Development priorities and versioned guarantees

Four identifiers must remain explicit: manuscript version, software version and source identity, certificate schema, and semantic contract. Manuscript 2.0 accompanies the preparation of software 0.2.0a1; the coordinatewise schema and contract remain unchanged, while research capsules preserve their individual schemas and source fingerprints. Lean additionally pins its compiler version. A proposed release tag is not evidence that a remote publication has occurred.

The release manifest maps K1–K10 to code, saved proof data, and validation. The manuscript has no invented DOI. If it is deposited later, a version DOI should identify this exact text and a concept DOI the evolving record. The public repository should preserve the earlier paper/software map and identify the newly released commit after publication.

The next research priority is the complete joint-carrier argument for Graal `create`. It requires deriving helper preconditions from reachable caller states, preserving temporary masks and empty outcomes, and proving the stated stabilization behavior. In particular, the lower sign condition must be established where it is used; it cannot be assumed because it made a standalone helper theorem true.

The next formalization step can reuse the general gluing theorem for the 49-state descending certificate, then address the outer order cases and source correspondence. Extending the algebraic Lean carrier beyond $\mathcal{P}(\{0,1\})$ is useful when a concrete next proof requires it. Formalizing the whole series at once is not a prerequisite for checking a stable applied fragment.

The global algorithmic challenge is observation inference. Current producers automatically complete rows, search closed carriers, and check proposed certificates within designed observation families. They do not yet derive a suitable finite family for any source program. A stronger engine should derive candidate observations and their sufficient domains from consumer obligations, retain separating countermodels when a factor loses essential information, and report exhaustion explicitly.

Wider application also requires target adapters matching actual language preconditions, integration of the research interfaces into a stable public API, and controlled comparisons on new external sources. Coverage, certificate size, discovery effort, replay effort, native correspondence, and proof-assistant coverage are distinct progress measures. Each should retain its own denominator and failure outcomes.

13 Relation to previous work

Nice to Meet You synthesizes practical MLIR abstract transformers and combines components by abstract meet [4, 5]. Its candidate validation uses Z3 over finite widths. QKF studies a parametric certificate route: source reconstruction and replay connect the program to a semantic interface, and a finite product or gluing argument lifts the interface check to every positive width. The original artifact, the total research contract, and LLVM’s guarded or poison-producing operations require explicit semantic correspondence when their verdicts are compared.

KnownBits and the related tristate-number domain are established representations [6, 7]. Their arithmetic properties motivate concrete native facts in this study. The contribution here concerns the specified finite semantic classes, source-bound proof architecture, and the use of consumer observations and forced rows to compose word-level obligations. The fixed-point and finite-state checks themselves use standard finite algorithms.

Bit-width-independent SMT approaches seek proofs through integer arithmetic, uninterpreted functions, and quantified encodings [9]. QKF currently succeeds where an adequate finite observation carrier and its semantic bridge can be constructed. Alive2 addresses LLVM refinement through bounded translation validation [10]; its source model and obligations differ from the total transfer-expression contract here. The experiments do not establish that one of these approaches subsumes the others.

The algebraic organization is inherited from Papers I and II [1, 2]. The comparable-pair principal-congruence formula used in Section 4 is due to Martin-Maroto, Ricciardo, and de Polavieja [8]. Kernel factorization, inverse-image preservation, and induction are elementary ingredients. The applied step is to justify a particular native observation, retain its labels and domain, complete its rows under the applicable theory, and make those rows support the checked composition.

Lean supplies an external proof checker for the stated pilot [12, 13]. This reduces the trusted mathematical argument for that fragment to its definitions, proof terms, and standard Lean assumptions. It does not automatically verify the surrounding parsers, Python implementations, or source-language bridges. The precise boundary is part of the result.

14 Conclusion

The project has progressed from a finite coordinatewise checker to a family of interfaces that retain the observations required by a consumer and compose them across an entire word. Forced rows provide a concrete connection to the first paper: supplied atomic values determine the tables actually used by the proof transitions, while kernels distinguish local information

loss from collapse of the protected carrier. Finite gluing connects those actions to arbitrary lengths once the numerical and source bridges are supplied.

The signed-bound case study exhibits both strength and limits. A shared-prefix carrier supports an exact upper maximum; a two-class carry factor supports a conditional lower-preservation argument. The lower result need not be exact, and its sign condition matters. The Lean pilot now checks the ascending mathematical fragment and the modeled cell-factor bridge.

The main open step is to make adequate observation selection and joint-carrier composition more general, then connect them to a stable implementation and stronger source verification. The present results provide concrete mechanisms, replayable evidence, countermodels to inadequate interfaces, and a machine-checked starting point for that work.

A Semantic operations and the complete rule catalogue

A.1 Word operations used by the frontend

Let $M = 2^w - 1$. Integer literals are reduced modulo 2^w , and signed values use two's-complement interpretation. Bitwise NOT is $M \text{ XOR } x$. Addition, subtraction, and multiplication are modular. Unsigned and signed minima/maxima use their respective orders.

For unsigned shift count c , logical shifts return zero if $c \geq w$; arithmetic right shift fills with the sign bit, including for $c \geq w$. Clearing or setting c low/high bits affects $\min(c, w)$ positions. Sign-bit operations affect position $w - 1$. Leading/trailing zero/one counts are the number of consecutive matching bits from the chosen end, in $[0, w]$.

Unsigned division/remainder by zero return M and the dividend, respectively. Signed division truncates toward zero and is then reduced modulo width; division by zero returns one for a negative dividend and M otherwise. Signed remainder has the dividend's sign and returns the dividend at divisor zero. Comparisons `cmp0` through `cmp9` denote, in order, equality, inequality, signed $<$, $\leq$, $>$, $\geq$, and unsigned $<$, $\leq$, $>$, $\geq$. Boolean connectives operate on Boolean values; selection chooses a word branch by a Boolean condition. Pair construction and projection have their ordinary meanings.

These definitions specify a pure total word language. They are not a claim that an arbitrary LLVM/MLIR backend implements this contract. The exact contract identifier and source grammar are part of the release manifest.

A.2 All 29 identifiers

In the table, 0 and $\top$ denote zero and all-ones words, W denotes the width word, and 1 denotes the ordinary integer-one word. An identity with two operand orders is used only when the implementation permits both. Guards are those of Lemma 6.2.

Identifier	Accepted law or condition
<code>idempotent</code>	$x \circ x = x$ for bitwise AND/OR and signed/unsigned min/max.
<code>self-cancel</code>	$x \text{ XOR } x = 0$ and $x - x = 0$.
<code>zero-and</code>	AND or multiplication with either operand zero gives zero.
<code>ones-or</code>	OR with either operand all-ones gives all-ones.
<code>zero-identity</code>	OR, XOR, and addition with zero on either side; subtraction with zero on the right.
<code>ones-and</code>	AND with all-ones on either side returns the other operand.
<code>not-constant</code>	NOT interchanges zero and all-ones.
<code>not-not</code>	Double bitwise NOT is identity.
<code>disjoint-and</code>	$x \& y = 0$ when the disjoint-support guard succeeds.

Identifier	Accepted law or condition
<code>disjoint-add</code>	$x + y = x \mid y$ under the same guard.
<code>clear-width</code>	Clearing W low or high bits gives zero.
<code>count-constant</code>	A matching bit count on a constant word is W ; a nonmatching count is zero.
<code>clear-zero-count</code>	Clearing zero low/high bits is identity; clearing bits from zero gives zero.
<code>set-zero-count</code>	Setting zero low/high bits is identity.
<code>set-leading-included</code>	Setting <code>countl_one(x)</code> high bits of y is identity when support of x is included in support of y .
<code>umin-zero</code>	Unsigned minimum with zero gives zero.
<code>umax-ones</code>	Unsigned maximum with all-ones gives all-ones.
<code>remainder-zero</code>	Unsigned or signed remainder of zero gives zero, including a zero divisor.
<code>urem-one</code>	Unsigned remainder by integer one gives zero.
<code>udiv-one</code>	Unsigned division by integer one is identity.
<code>count-bound</code>	$W <_u \text{count}(x)$ is false and $W \geq_u \text{count}(x)$ is true.
<code>compare-reflexive</code>	Reflexive comparison is true for eq, signed/unsigned $\leq, \geq$; false for ne and strict comparisons.
<code>compare-unsigned-bound</code>	Strict comparisons beyond unsigned endpoints are false; inclusive comparisons from zero or to all-ones are true. Also $0 <_u x$ and $x >_u 0$ are true if the low-bit guard proves one.
<code>select-constant</code>	A constant Boolean condition selects its corresponding branch.
<code>select-same</code>	Equal branches remove a selection.
<code>bool-constant</code>	Evaluate two Boolean constants; false absorbs Boolean AND and true absorbs Boolean OR.
<code>set-low-one</code>	Setting one low bit of zero returns integer one.
<code>zero-shift</code>	Shifting zero in any supported direction returns zero.
<code>shift-zero-count</code>	Shifting by zero is identity.

A.3 Proof arguments by family

Bitwise identities follow separately at every bit; modular cancellation and zero multiplication follow in $\mathbb{Z}/2^w\mathbb{Z}$. Signed and unsigned min/max are idempotent because each is a minimum or maximum in a total order. No signed-order law is substituted for an unsigned-order law.

Disjoint addition is ordinary addition without carries: at each position at most one operand bit is one, and induction from carry zero keeps carry zero. Reduction modulo 2^w then gives OR. In the leading-bit rule, every leading one of x is already a one of y by support inclusion; setting these positions changes no value. These arguments use proved guards rather than sampled word instances.

The clear/set/count rules follow directly from the affected position sets and the definitions of counts. Every count is at most w , and the encoded width word is its exact unsigned integer value. The integer-one rules remain valid at width one. That does not make arbitrary uses of integer one coordinatewise.

Unsigned comparisons with zero and M use the extrema of B_w . An operand with low bit one is nonzero, hence strictly positive in unsigned order. Reflexive comparisons and Boolean/selection identities follow from their truth tables. Remainder of zero is zero for nonzero divisors and, by the specified total extension, also for zero divisors. Division or unsigned remainder by one has the indicated value. The shift identities also cover oversized counts because the shifted zero's sign is zero.

The rule catalogue specifies local semantic equalities. Checking that the implementation tests the matching syntax and side conditions is part of the code audit and release regressions.

An altered rule set requires a renewed correspondence check before applying the old source-to-verdict theorem to a new executable.

References

- [1] L. Shcherbakov. *Initial semantics of incompletely specified actions: forced quotients, kernel saturation, and completion*. QKF project, Paper I, preprint v1.0, manuscript revision 11 September 2026. Companion manuscript; no DOI assigned in this package.
- [2] L. Shcherbakov. *Equality-visibility depth in equational presentations: finite-ground spectra, certificates, and semigroup space*. QKF project, Paper II, preprint v1.0, manuscript revision 12 September 2026. Companion manuscript; no DOI assigned in this package.
- [3] L. Shcherbakov. *QKF Certifier: Width-independent KnownBits certificates*. Software v0.1.0a1, 2026. MIT license. <https://github.com/Len989/QKF-Certifier/tree/f93561682319e8b911ed32c01583f2a496dc59fc>.
- [4] X. Peng, D. Kennedy, Y. Fan, B. Greenman, J. Regehr, and L. D’Antoni. *Nice to Meet You: Synthesizing Practical MLIR Abstract Transformers*. POPL, 2026. <https://doi.org/10.1145/3776722>. Preprint: <https://arxiv.org/abs/2512.06442>.
- [5] X. Peng et al. *Artifact for Nice to Meet You: Synthesizing Practical Abstract Transformers for MLIR*. Version v2 (Updated README), 2025. <https://doi.org/10.5281/zenodo.17371668>. Example provenance: <https://github.com/Hatsunespica/xdsl-smt/tree/4ec509edab067d62c00f76778789170061ae299c>.
- [6] LLVM Project. *llvm::KnownBits structure reference*. Official documentation, accessed 12 September 2026. https://llvm.org/doxygen/structllvm_1_1KnownBits.html.
- [7] H. Vishwanathan, M. Shachnai, S. Narayana, and S. Nagarakatte. *Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers*. CGO, 2022. <https://doi.org/10.1109/CGO53902.2022.9741267>. <https://arxiv.org/abs/2105.05398>.
- [8] F. Martin-Maroto, A. Ricciardo, and G. G. de Polavieja. *The pairwise distributive law of semilattice congruences*. Algebra Universalis 87, article 29, 2026. <https://doi.org/10.1007/s00012-026-00942-x>.
- [9] A. Niemetz, M. Preiner, A. Reynolds, Y. Zohar, C. Barrett, and C. Tinelli. *Towards Bit-Width-Independent Proofs in SMT Solvers*. CADE, 2019. <https://arxiv.org/abs/1905.10434>.
- [10] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. *Alive2: Bounded Translation Validation for LLVM*. PLDI, 2021. <https://doi.org/10.1145/3453483.3454030>.
- [11] Graal Project. *IntegerStamp.java*. Pinned source revision 3f5efeb49d934f8a0bcb3c115a6a28caa5622975. <https://github.com/oracle/graal/blob/3f5efeb49d934f8a0bcb3c115a6a28caa5622975/compiler/src/jdk.graal.compiler/src/jdk/graal/compiler/core/common/type/IntegerStamp.java>.
- [12] L. de Moura and S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE, 2021. https://doi.org/10.1007/978-3-030-79876-5_37.
- [13] Lean FRO. *The Lean Language Reference: axioms and proof validation*. Toolchain used: Lean 4.33.0. <https://lean-lang.org/doc/reference/latest/Axioms/>. Release: <https://github.com/leanprover/lean4/releases/tag/v4.33.0>.